Doom Update

3/30/2026

Agenda

Execution Algorithm [01]

Genetic Algorithm [02]

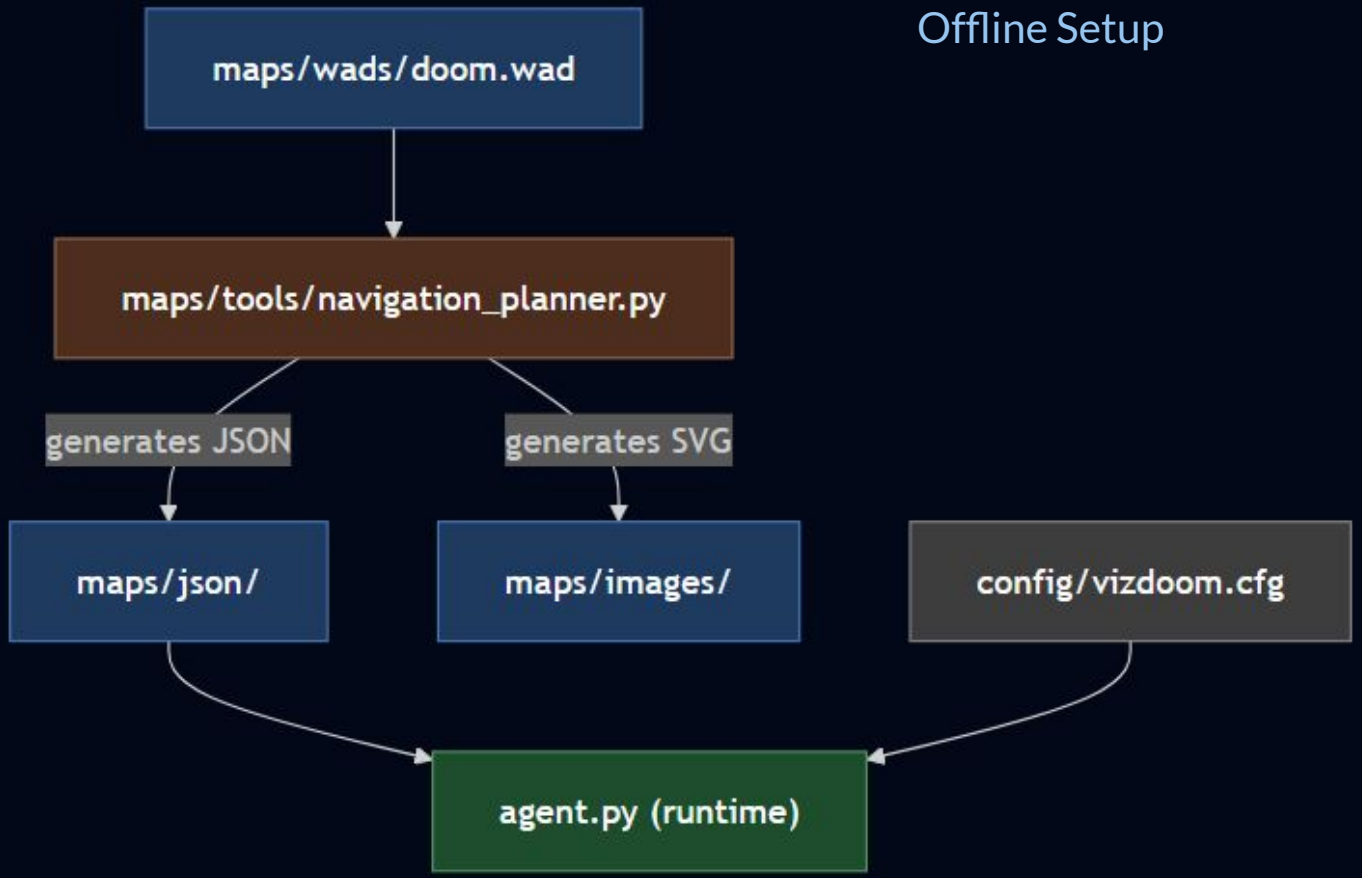
Telemetry [03]

Codebase Improvements [04]

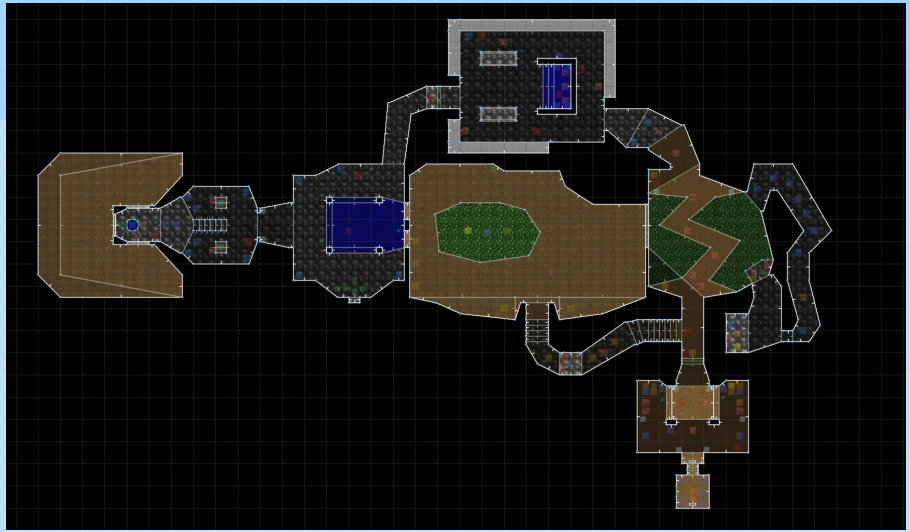
Future Work [05]

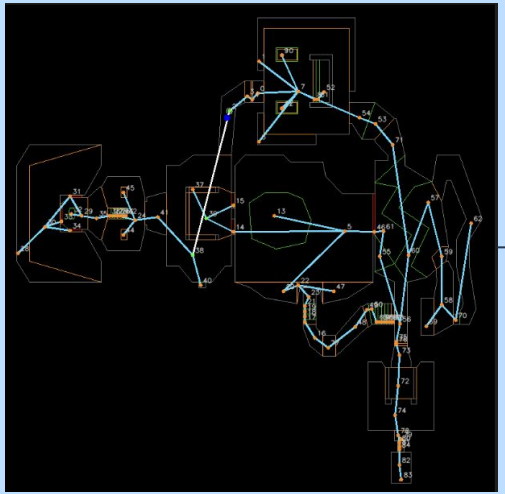
Execution Algorithm

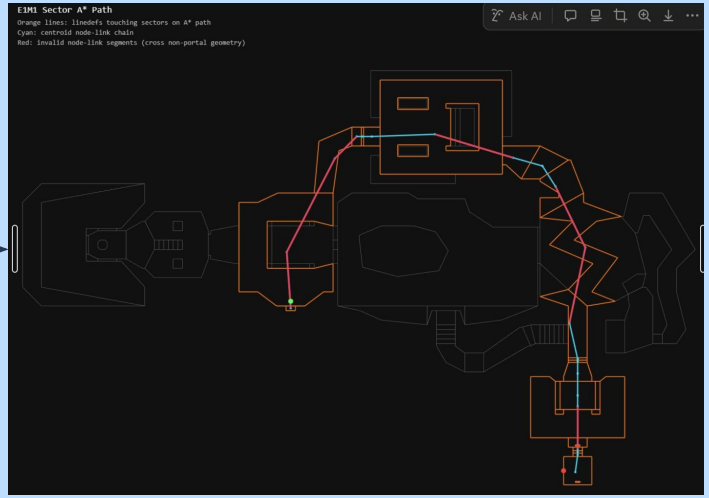
Offline Setup



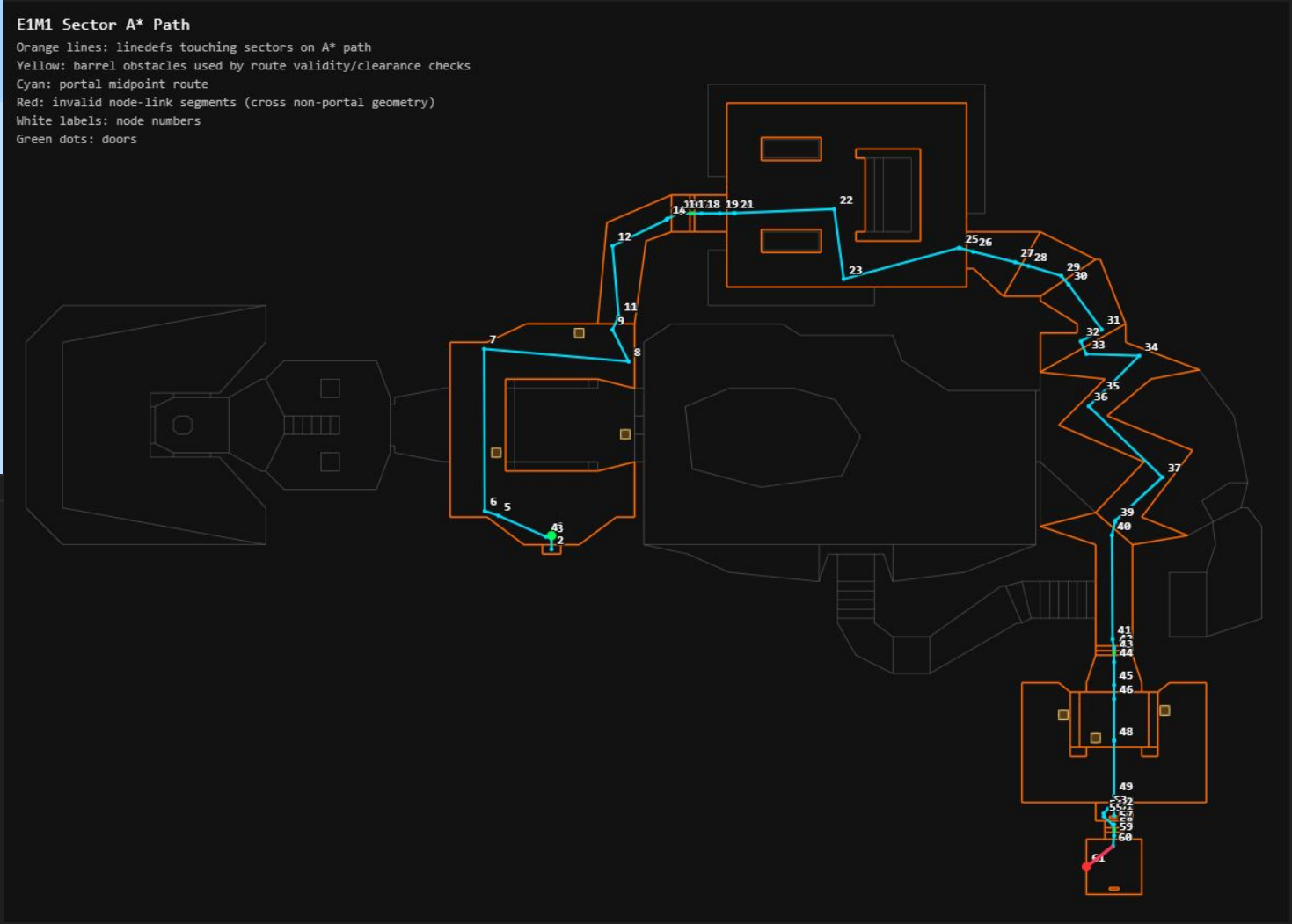
Static Node Map Creation





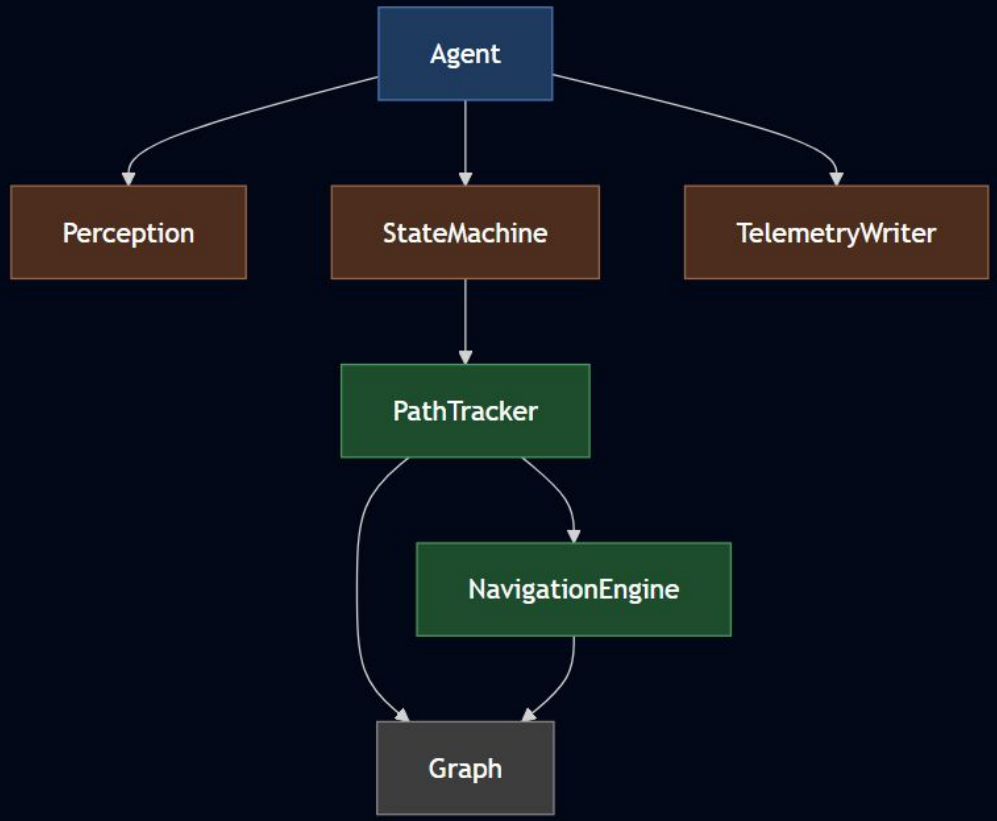


Static Node Map Creation

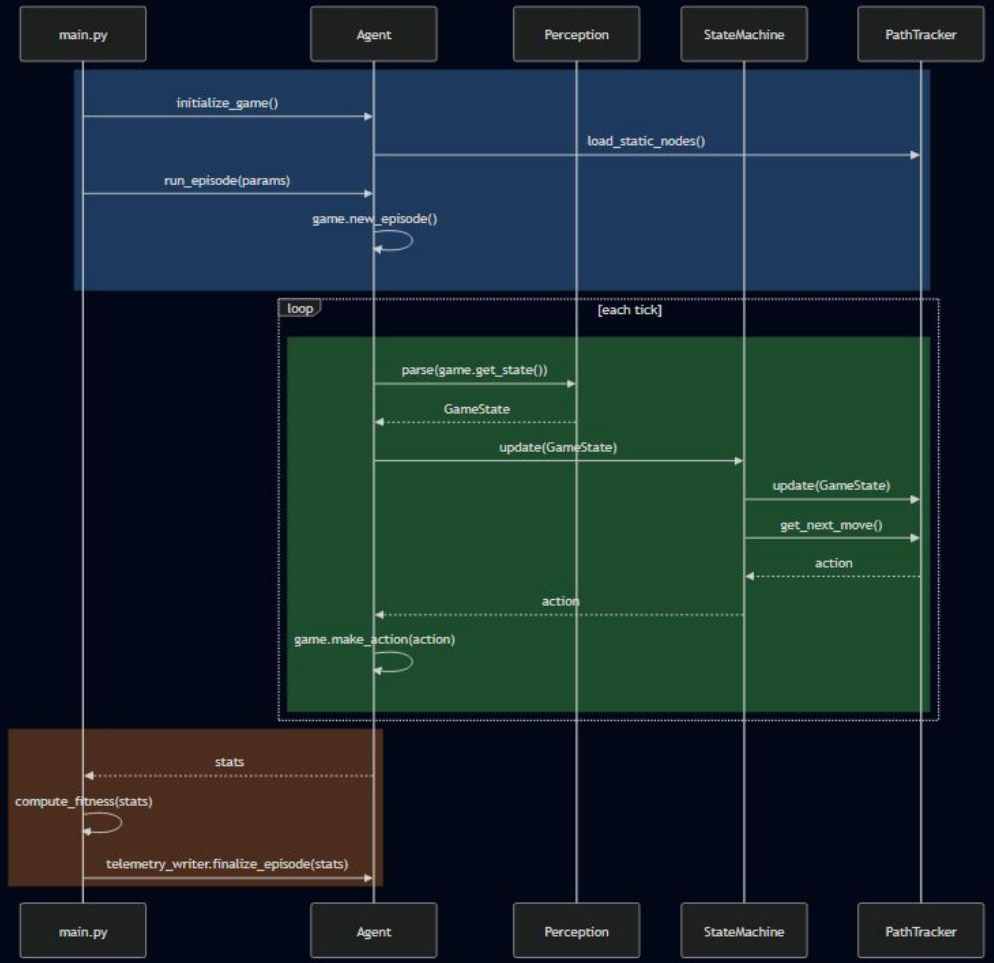



```
{
  "wad": "maps/wads/doom.wad",
  "map": "E1M1",
  "node_points": [
    {
      "x": 1056.0,
      "y": -3616.0,
      "type": "waypoint",
      "special": null
    },
    {
      "x": 1056.0,
      "y": -3664.0,
      "type": "waypoint",
      "special": null
    },
    {
```

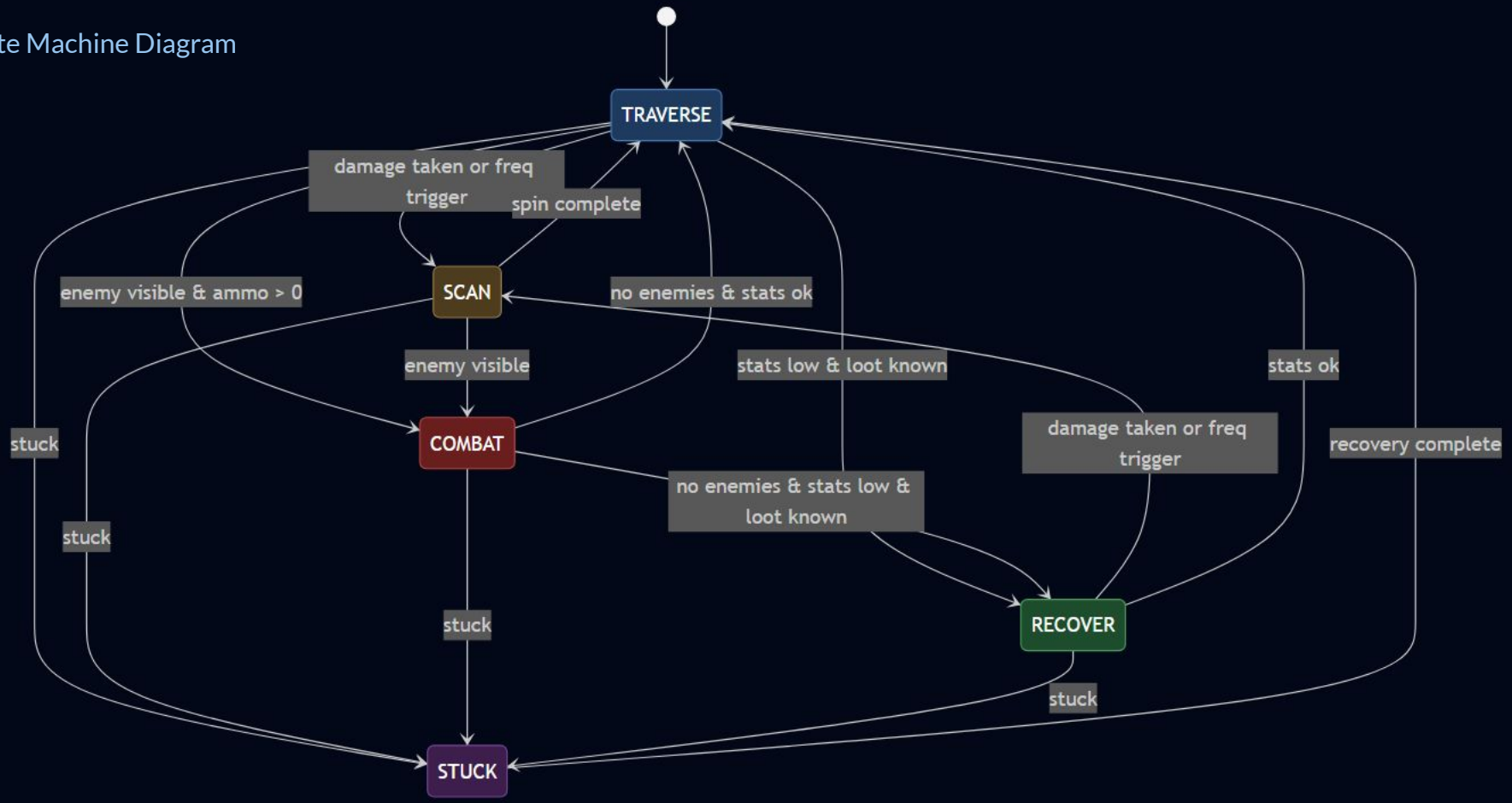
Runtime Class Ownership Diagram



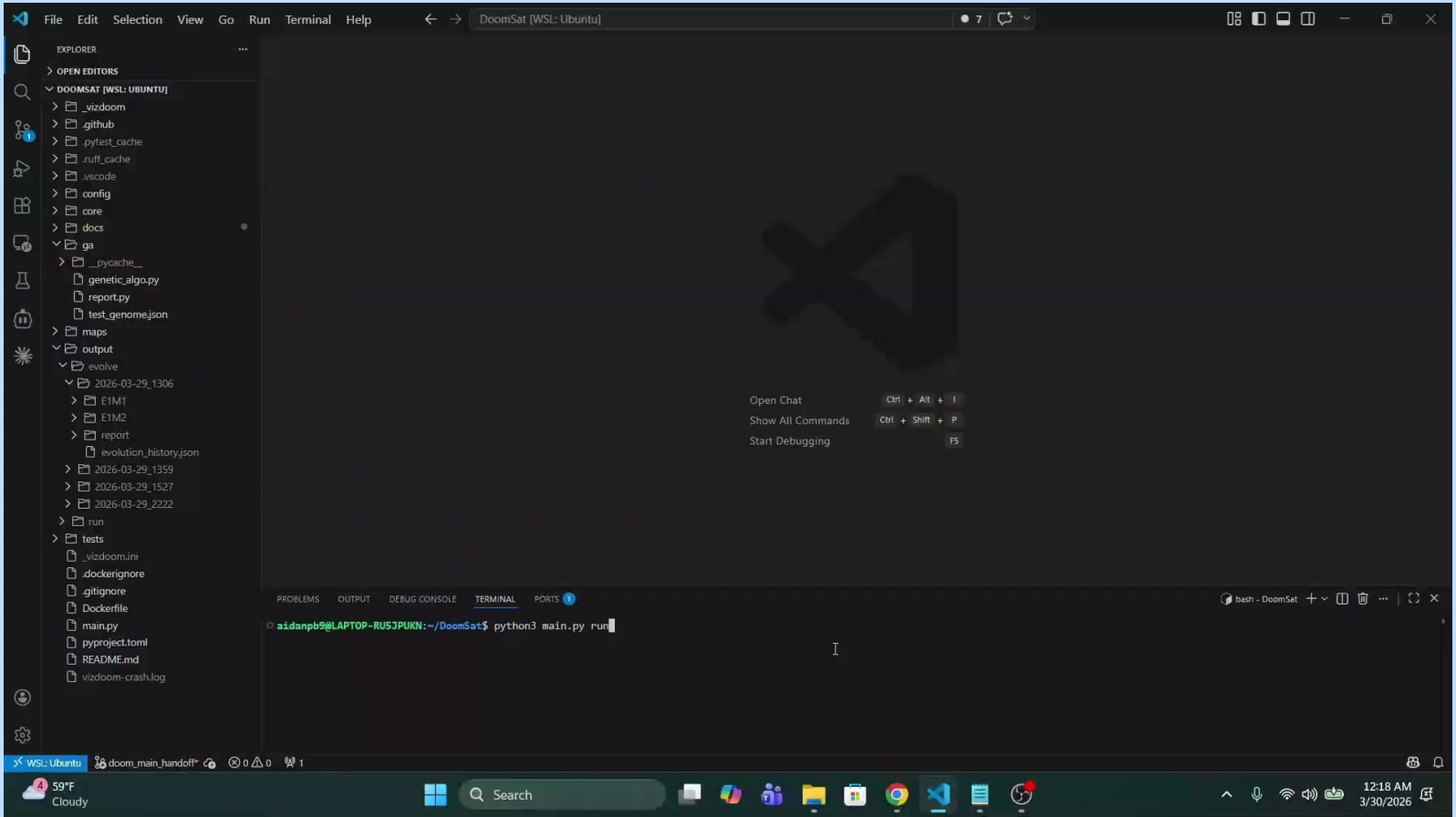
Runtime Sequence Diagram



State Machine Diagram



Gameplay


File Edit Selection View Go Run Terminal Help
DoomSat [WSL: Ubuntu]
EXPLORER
OPEN EDITORS
DOOMSAT [WSL: UBUNTU]
_vizdoom
.github
.pytest_cache
.ruff_cache
.vscode
config
core
docs
ga
__pycache__
genetic_algo.py
report.py
test_genome.json
maps
output
evolve
2026-03-29_1306
E1M1
E1M2
report
evolution_history.json
2026-03-29_1359
2026-03-29_1527
2026-03-29_2222
run
tests
_vizdoom.ini
.dockerignore
.gitignore
Dockerfile
main.py
pyproject.toml
README.md
vizdoom-crash.log
Open Chat Ctrl + Alt + I
Show All Commands Ctrl + Shift + P
Start Debugging F5
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bash - DoomSat
aidanpb9@LAPTOP-RU5JPUKN:~/DoomSat$ python3 main.py run
WSL: Ubuntu
doom_main_handoff*
59°F
Cloudy
Search
12:18 AM
3/30/2026

Genetic Algorithm

2-Agent Micro-Population Steady-State Elitist Genetic Algorithm

Elite: Current best parameter set

Challenger: Mutated derivative of the elite

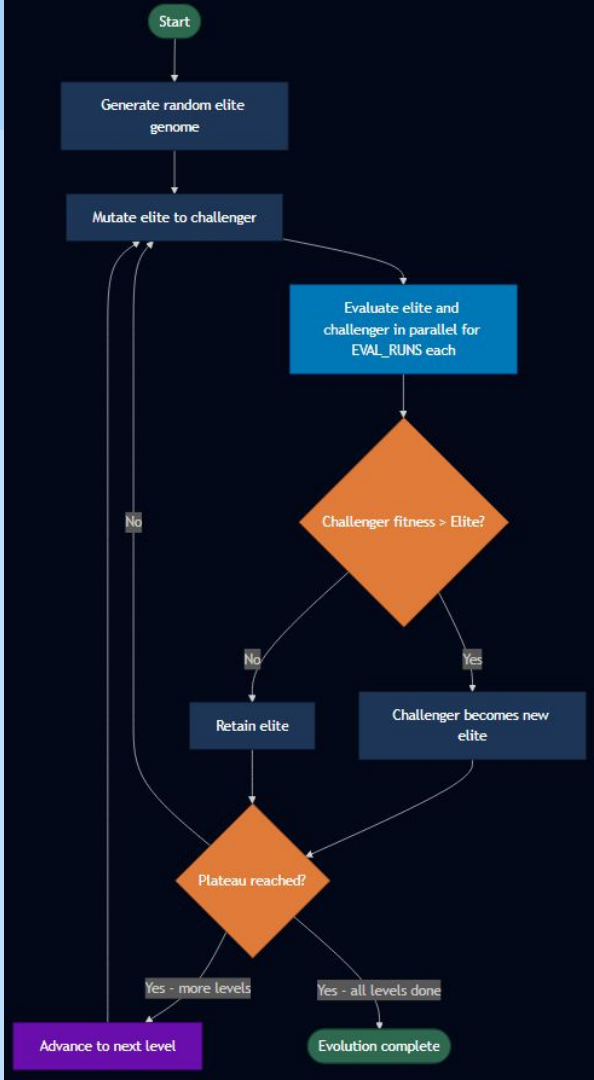
Only one agent changes per generation.

Elite is never discarded unless beaten.

Winner becomes elite for the next generation.

Evaluate best genome per-level (high level variance).

Each agent gets its own hardware process (multiprocessing).



Fitness Criteria

- Heavily favor completion
- Value completion time. Value health > armor > ammo.
- Provide feedback for agent to improve when not completing level

```
if level_completed:
    fitness = 5000                          #Base completion bonus
            + 500 * (1 - time_ticks / 4200) #Speed bonus
            + 2 * health_remaining          #Health
            + 1 * armor_remaining           #Armor
            + 0.5 * ammo_remaining          #Ammo
else:
    fitness = 0
            + 5 * enemies_killed            #Partial credit for progress
            + 10 * waypoints_reached        #Proximity to goal
```

Parameters

Hyperparameters

Hyperparameter	Value	Description
Population size	2	Elite + challenger only
Eval runs	5	Episodes per genome per generation, averaged to reduce VizDoom RNG variance
Radiation intensity	0.25	Probability per parameter of a bit-flip mutation occurring
Plateau generations	10	Generations without elite change before advancing to next level
Episode timeout	12600 ticks (360 seconds)	E1M1 time limit
Evaluation seed	Random per episode	Python RNG seeded fresh each episode, seed recorded in Tier 1 Telemetry

Evolvable Parameters (keeping their ranges wide for observation)

Exploration Parameters:

Parameter	Range	Description
`loot_node_max_distance`	200-1000 units	Distance from agent that loot nodes are placed
`stuck_recovery_ticks`	35-140 ticks	Ticks of turn+forward to dislodge from obstacles

Combat Parameters:

Parameter	Range	Description
`combat_hold_ticks`	5-50 ticks	Ticks that agent stays in combat when enemy leaves FOV

Recovery Parameters:

Parameter	Range	Description
`health_threshold`	0-100	Health level triggering RECOVER state
`armor_threshold`	0-100	Armor level triggering RECOVER state
`ammo_threshold`	0-200	Ammo level triggering RECOVER state

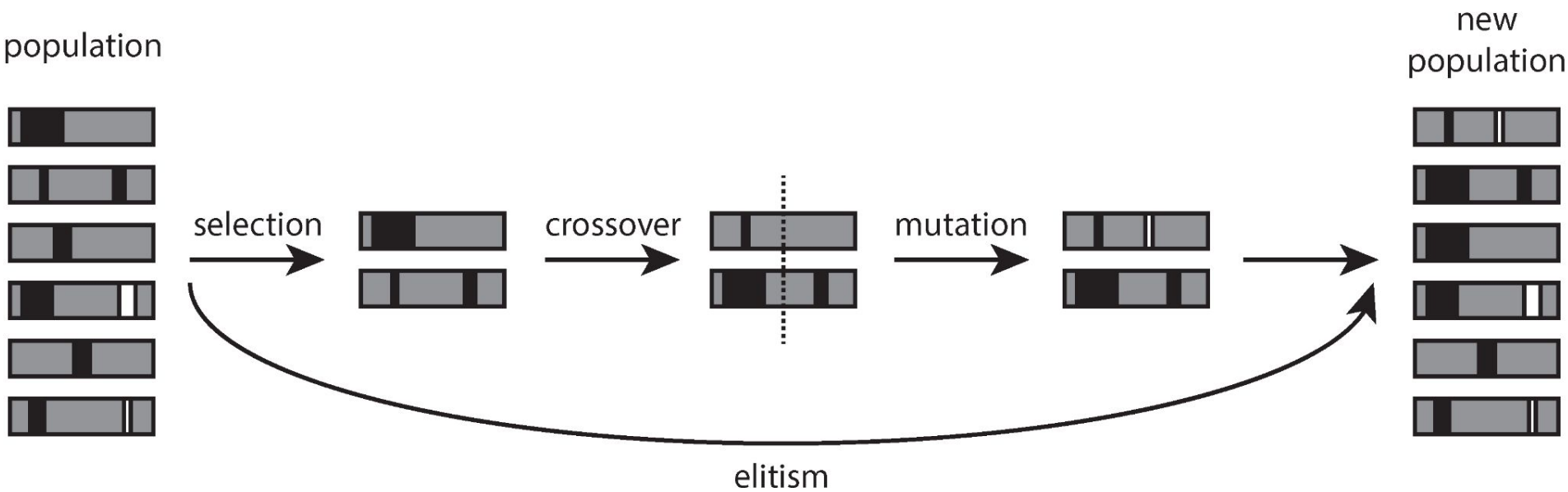
Scan Parameters:

Parameter	Range	Description
`scan_interval`	70-420 ticks (2-12sec)	How often agent is likely to scan

Total: 7 parameters per genome

Why not a crossover GA?

- Only one spacecraft flight system, population size of 2 is a hardware constraint
- Crossover requires many parents, with 2 parents it's a mutation
- Mutation represents how bit-flips affect the spacecraft, crossover does not



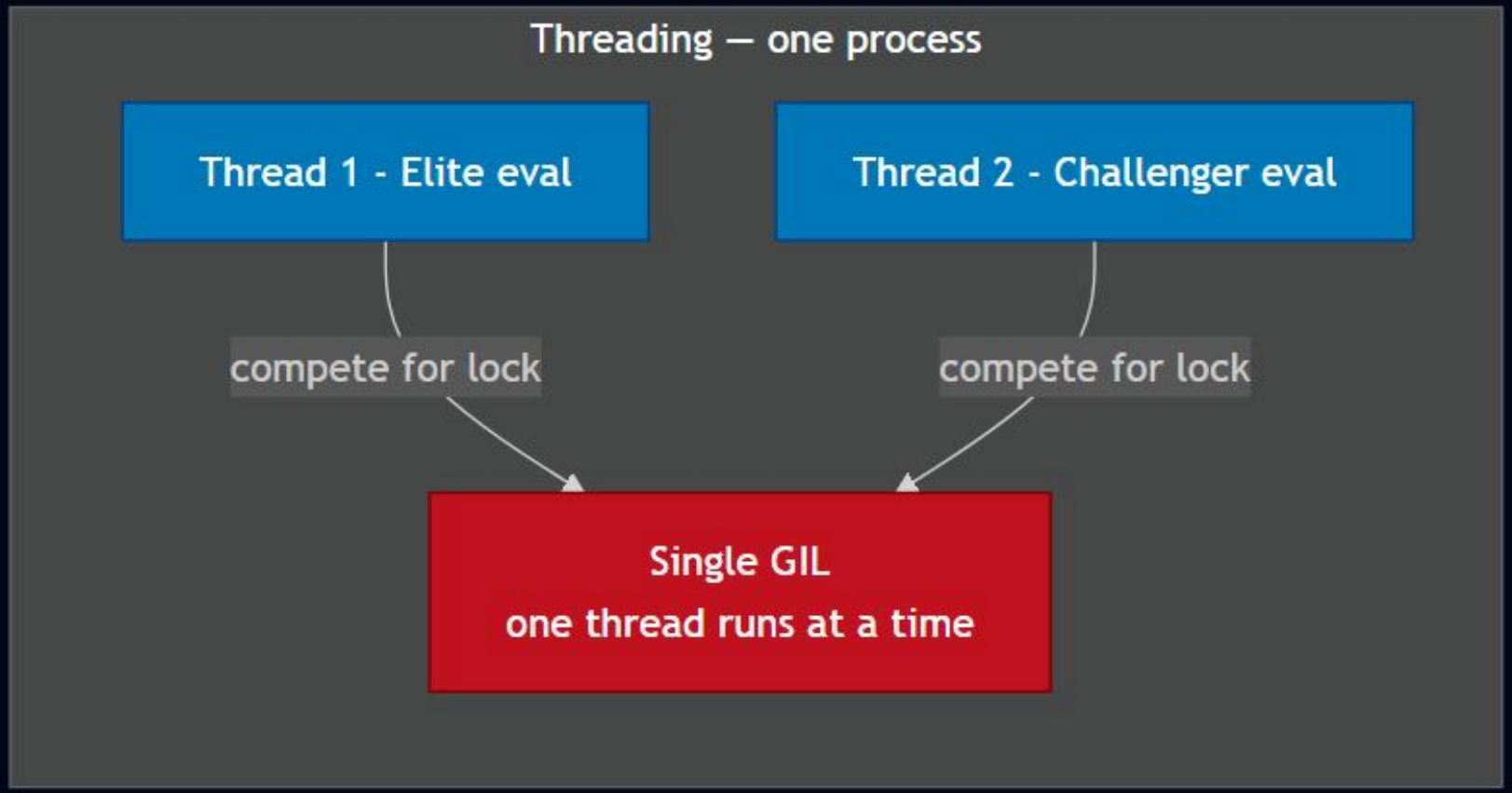
Constraints

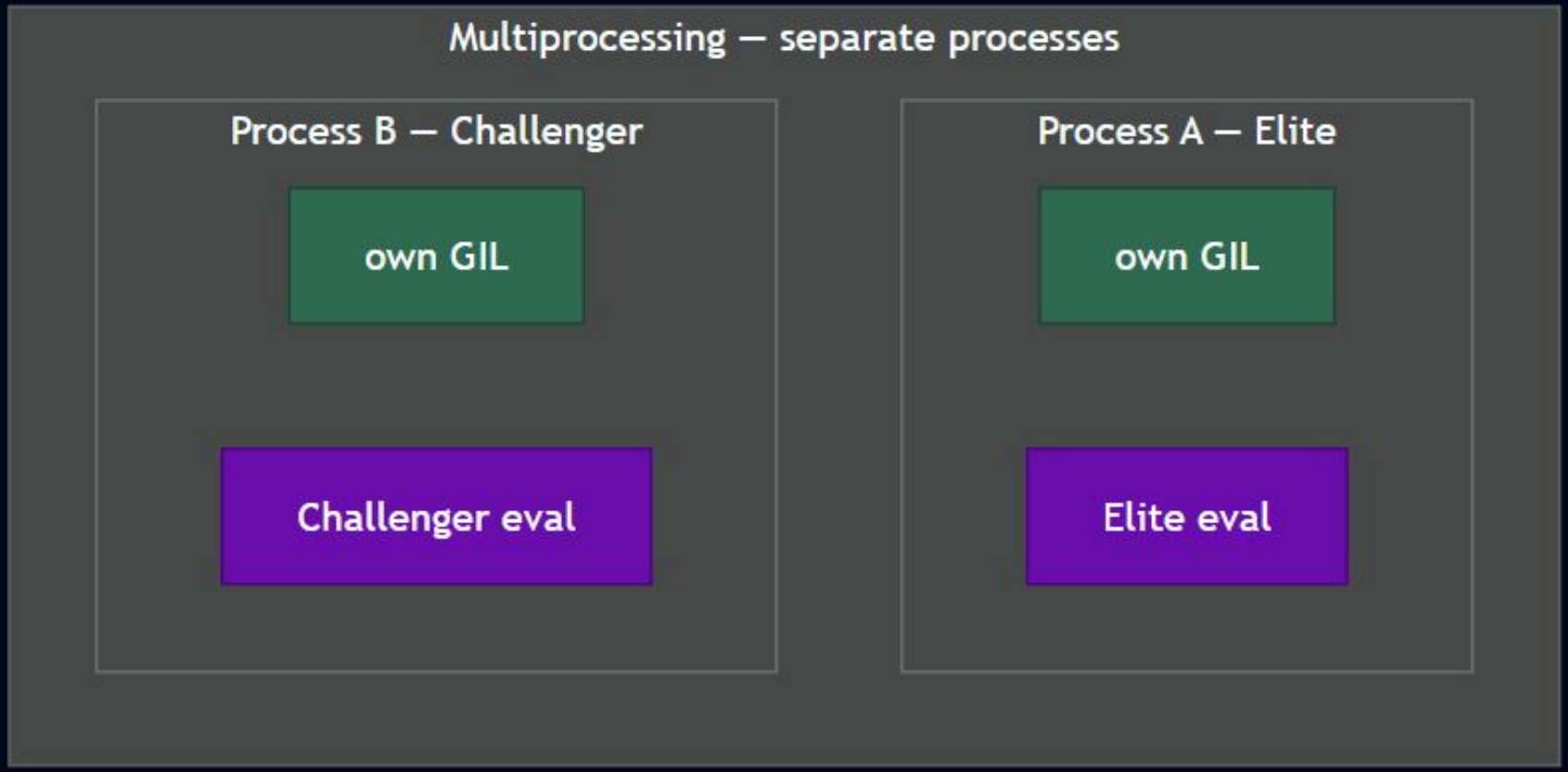
Headless mode:

- Windowed mode limited to 35 ticks/sec (several minutes / run)
- Headless (GA) capped by CPU (10 seconds per run)
- Actions still evaluated every tick

Seeds:

- Python RNG seed affects params. Leave random to avoid overfitting.
- Vizdoom seed controls enemy behavior
    - Constant in a generation (elite & challenger get the same)
    - Varies across generations


Threading — one process
Thread 1 - Elite eval
Thread 2 - Challenger eval
compete for lock
compete for lock
Single GIL
one thread runs at a time


Multiprocessing — separate processes
Process B — Challenger
own GIL
Challenger eval
Process A — Elite
own GIL
Elite eval

Parent Process
Module Globals
HEALTH_THRESHOLD = 50
SCAN_INTERVAL = 235
...
own GIL
fork() - copy-on-write
fork() - copy-on-write
Worker A — Elite
Module Globals
SCAN_INTERVAL = 235
...
HEALTH_THRESHOLD = 15
own GIL
Worker B — Challenger
Module Globals
SCAN_INTERVAL = 235
...
HEALTH_THRESHOLD = 85
own GIL

Fork vs Spawn

When creating a new worker process, the OS can either:

- **Fork:** copy the entire parent process memory into a new process instantly. Known as copy-on-write: pages are shared until one process writes to them, then copied.
- **Spawn:** start a fresh Python interpreter from scratch and re-import all modules

Fork is ~0.4s faster per pool creation because spawn must re-`dlopen` the VizDoom `.so` shared library in each new worker. On Linux, fork is the default and is safe for this project.

Benchmarks 3 Runs

Method	2 parallel evals wall time	Speedup vs sequential
Sequential	~60s	1.0x
ProcessPoolExecutor + fork	~31s	~1.95x
ProcessPoolExecutor + spawn	~36s	~1.65x
threading	~35s	~1.7x

```python
import multiprocessing as mp
from concurrent.futures import ProcessPoolExecutor, TimeoutError
from concurrent.futures.process import BrokenProcessPool
```

```python
#Submit the two genomes to the workers and get returns
game_seed = random.randint(0, 2**31) #same seed for both workers for fair comparison
f_a = self._pool.submit(eval_worker, elite, level, gen, "elite", game_seed, run_dir)
f_b = self._pool.submit(eval_worker, challenger, level, gen, "challenger", game_seed, run_dir)
#VizDoom can hang at the C++ level with no Python exception raised.
#If a worker freezes past 5 minutes we skip the generation entirely.
#The elite is unchanged, the plateau counter is not incremented,
#and the next generation will retry with a fresh mutant.
try:
    a_fit, a_beat = f_a.result(timeout=300)
    b_fit, b_beat = f_b.result(timeout=300)
except (TimeoutError, BrokenProcessPool):
    #BrokenProcessPool means the C++ VizDoom process was killed at the OS level.
    #The pool is dead and must be recreated or all future submits will also raise.
    self._pool = ProcessPoolExecutor(max_workers=2, mp_context=mp.get_context("fork"))
    print(f"[{level}] gen={gen} worker crashed or timed out, skipping generation")
    continue
```

Runtime Console Output

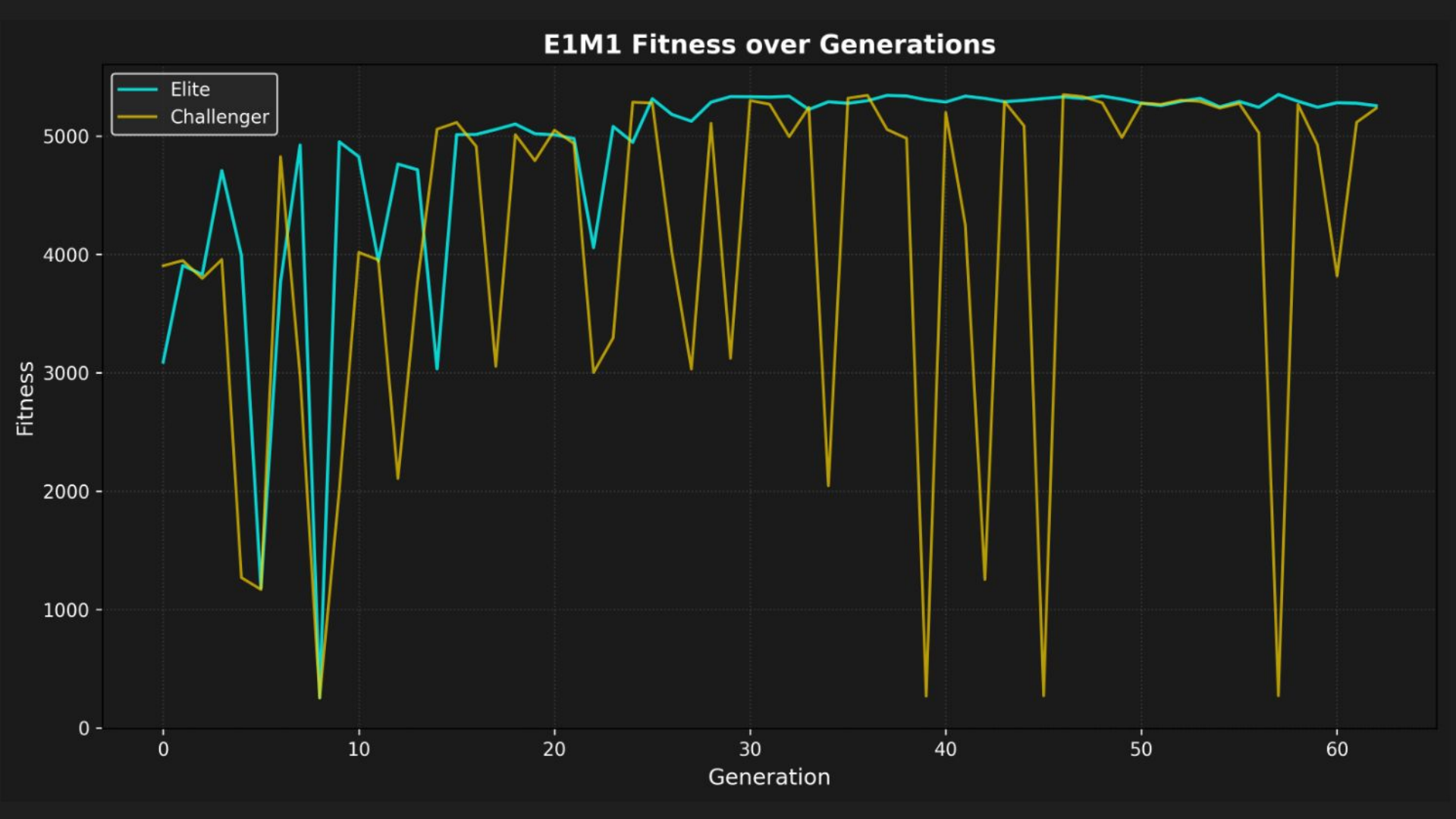
```
[E1M1] gen=12  elite=4762.69  challenger=2107.17  winner=elite  plateau=1/10
[E1M1] gen=13  elite=4714.81  challenger=3766.47  winner=elite  plateau=2/10
[E1M1] gen=14  elite=3032.83  challenger=5057.45  winner=challenger  plateau=0/10
[E1M1] gen=15  elite=5011.84  challenger=5114.35  winner=challenger  plateau=0/10
[E1M1] gen=16  elite=5013.14  challenger=4912.77  winner=elite  plateau=1/10
[E1M1] gen=17  elite=5055.21  challenger=3053.93  winner=elite  plateau=2/10
[E1M1] gen=18  elite=5101.09  challenger=5011.86  winner=elite  plateau=3/10
[E1M1] gen=19  elite=5019.05  challenger=4791.46  winner=elite  plateau=4/10
[E1M1] gen=20  elite=5009.91  challenger=5048.75  winner=challenger  plateau=0/10
[E1M1] gen=21  elite=4977.99  challenger=4935.85  winner=elite  plateau=1/10
[E1M1] gen=22  elite=4056.98  challenger=3001.84  winner=elite  plateau=2/10
[E1M1] gen=23  elite=5080.5  challenger=3294.25  winner=elite  plateau=3/10
[E1M1] gen=24  elite=4947.03  challenger=5285.54  winner=challenger  plateau=0/10
[E1M1] gen=25  elite=5314.64  challenger=5278.16  winner=elite  plateau=1/10
[E1M1] gen=26  elite=5182.27  challenger=4014.38  winner=elite  plateau=2/10
```

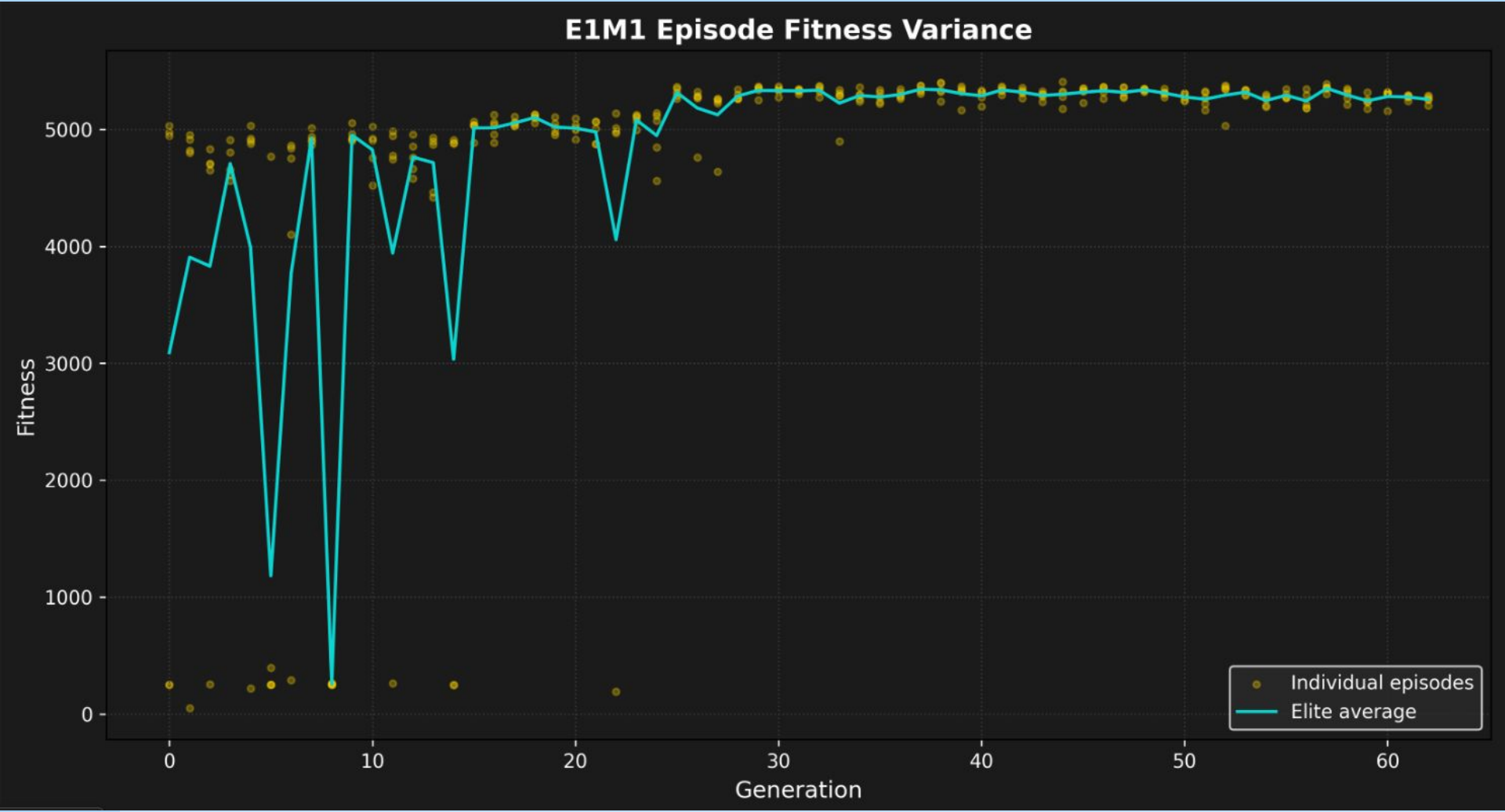
Final E1M1 genomes from 2 different evolutions

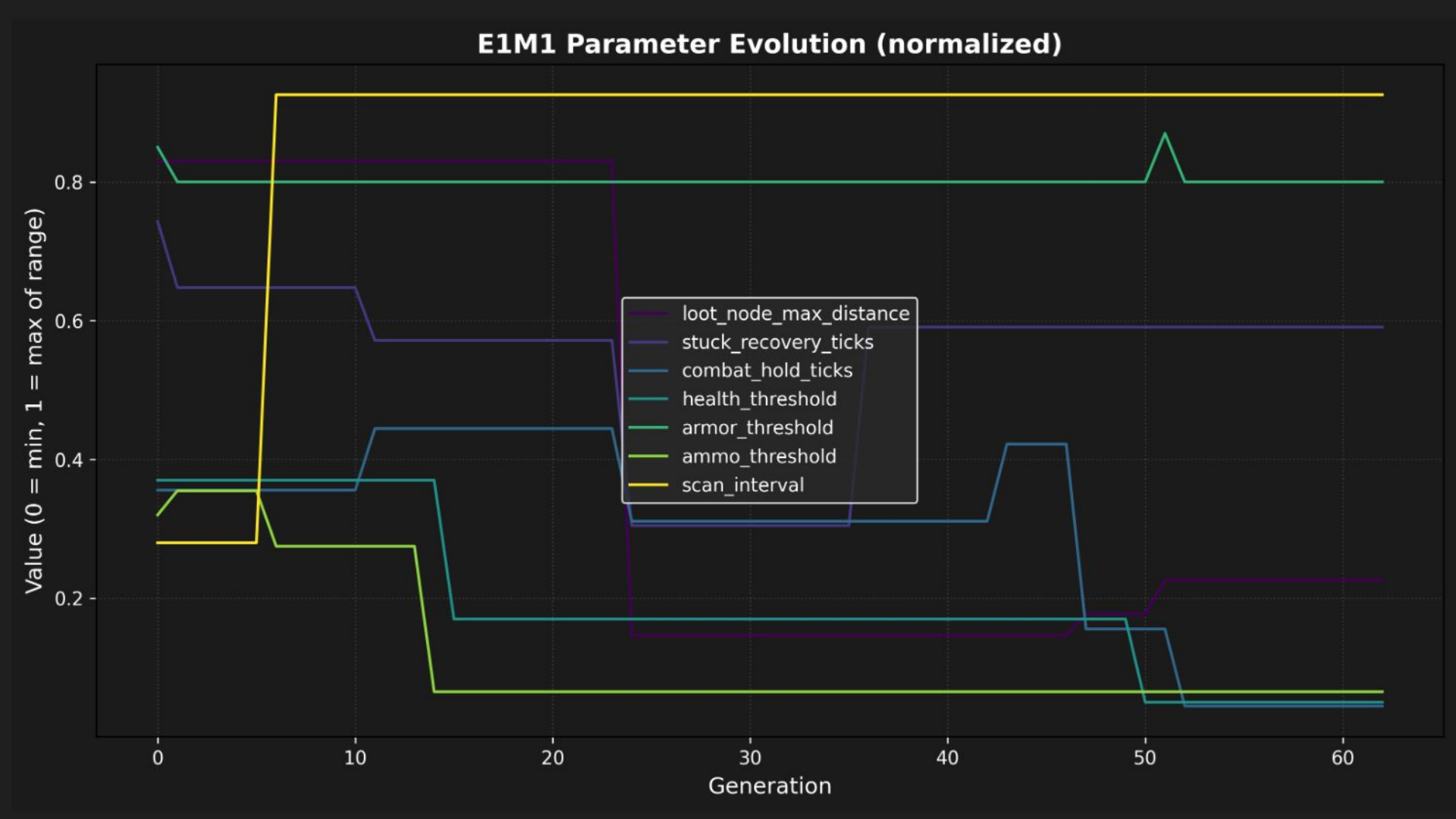

```
"42": {
  "elite_fitness": 4256.67,
  "challenger_fitness": 3679.51,
  "winner": "elite",
  "game_seed": 2056025480,
  "timestamp": "2026-03-29T13:42:10",
  "elite_genome": {
    "loot_node_max_distance": 234,
    "stuck_recovery_ticks": 112,
    "combat_hold_ticks": 13,
    "health_threshold": 30,
    "armor_threshold": 35,
    "ammo_threshold": 30,
    "scan_interval": 251
```

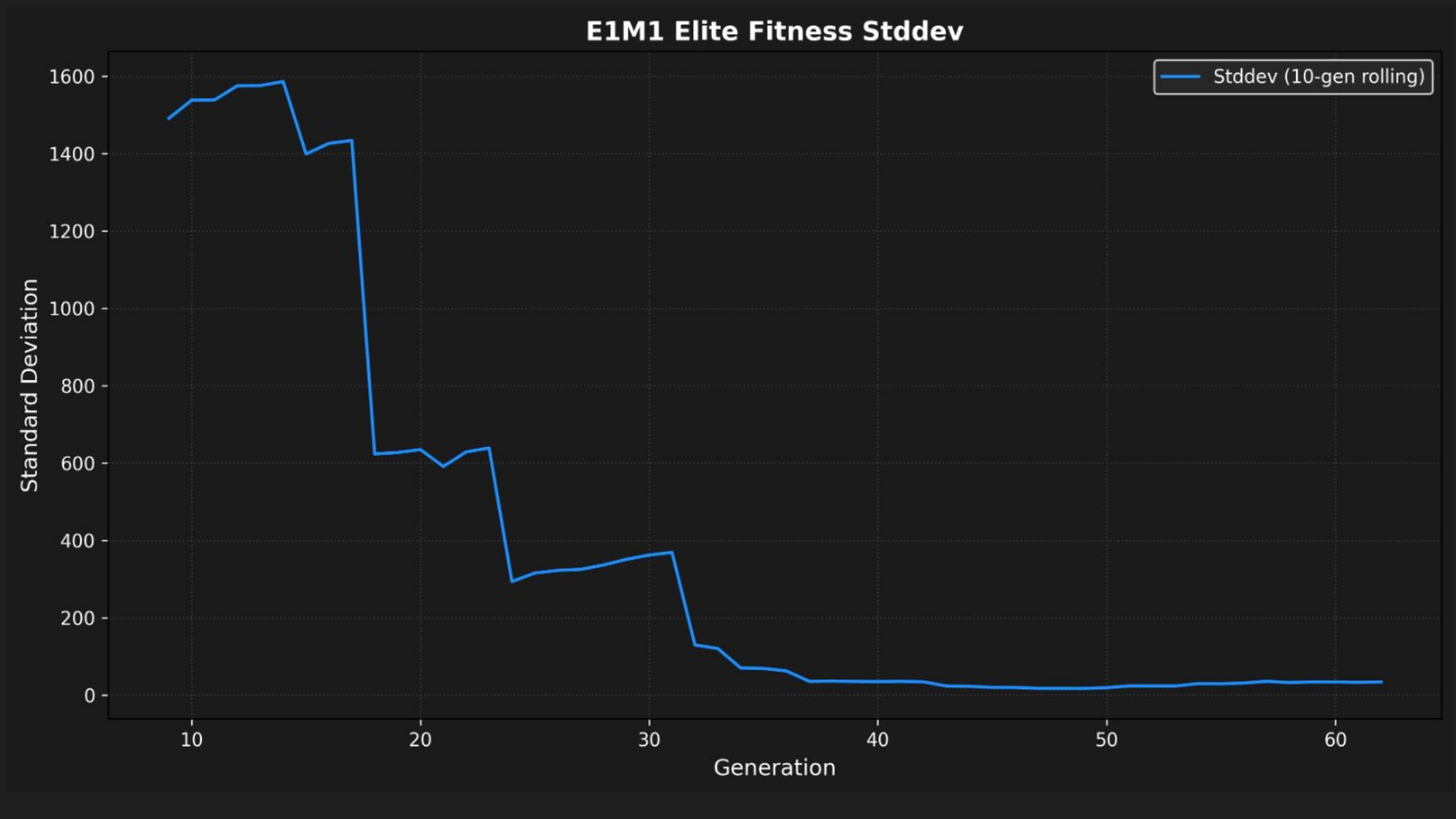

Ranges (1 sec=35 ticks)
200 -1000 units
35 - 140 ticks
5 - 50 ticks
0 - 100
0 - 100
0 - 200
70 - 420 ticks

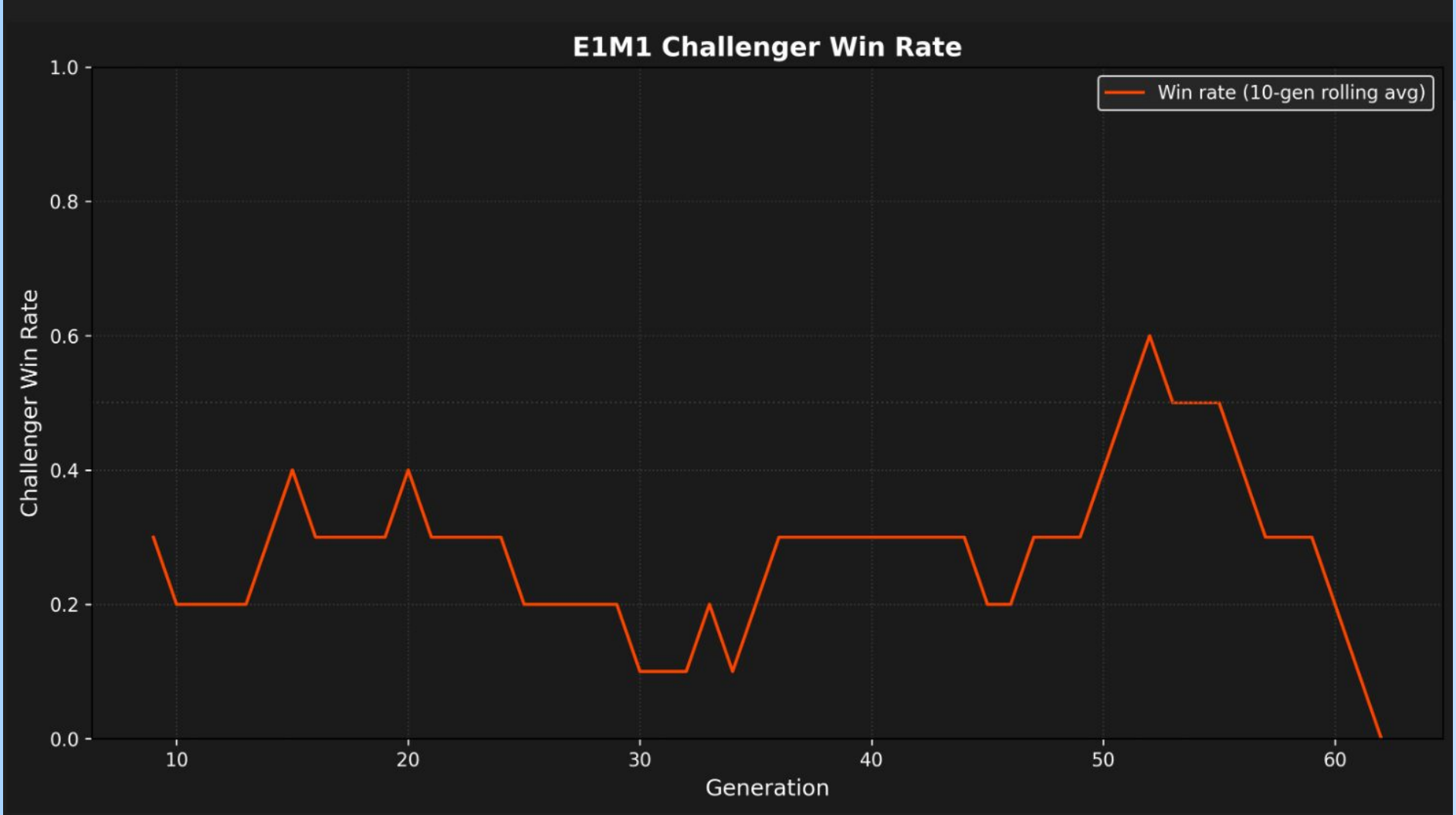

```
"62": {
  "elite_fitness": 5255.84,
  "challenger_fitness": 5234.58,
  "winner": "elite",
  "game_seed": 613114805,
  "timestamp": "2026-03-29T23:47:06",
  "elite_genome": {
    "loot_node_max_distance": 381,
    "stuck_recovery_ticks": 97,
    "combat_hold_ticks": 7,
    "health_threshold": 5,
    "armor_threshold": 80,
    "ammo_threshold": 13,
    "scan_interval": 394
```

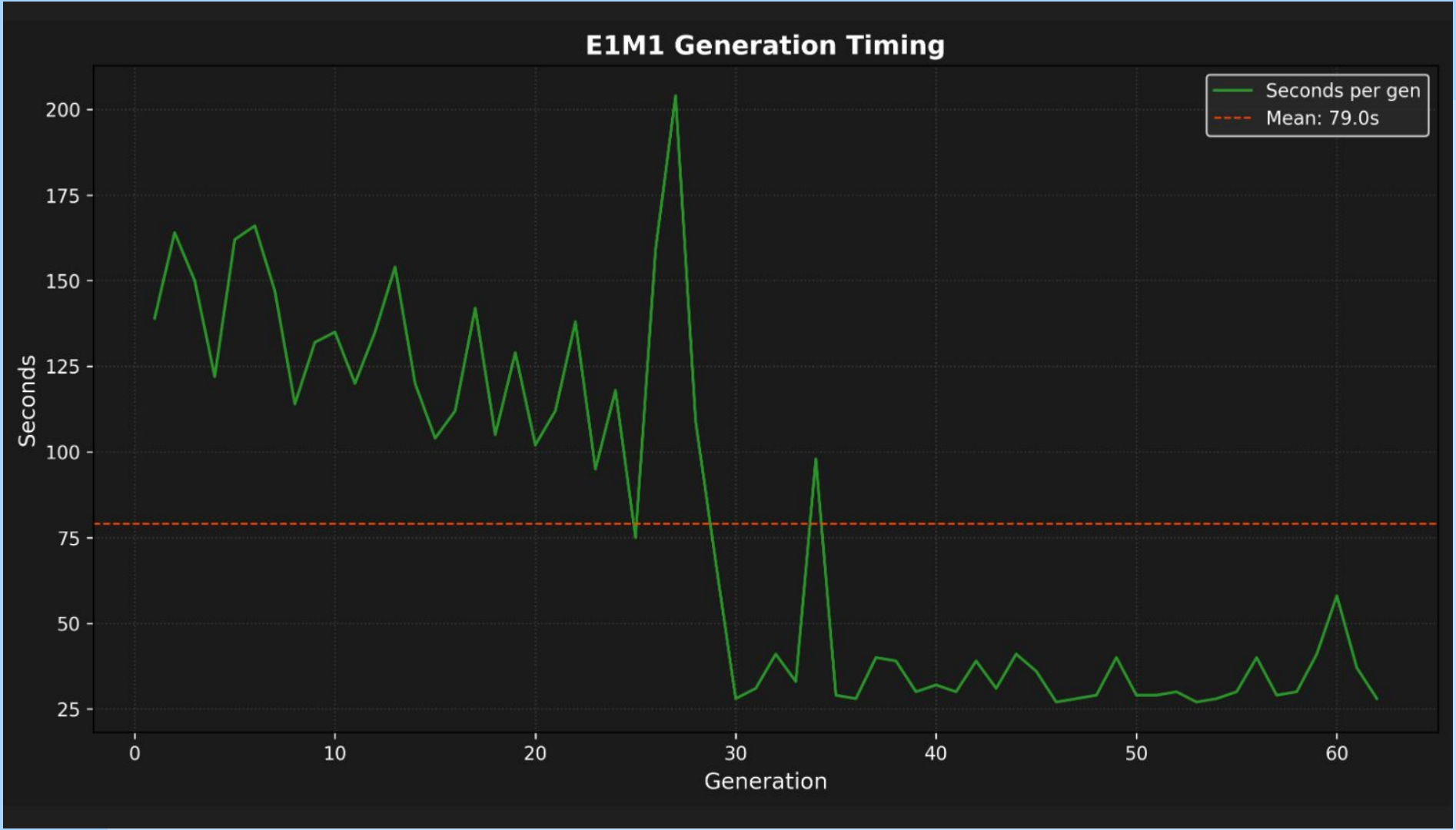

E1M1 Fitness over Generations
Elite
Challenger
Fitness
Generation
0
1000
2000
3000
4000
5000
0
10
20
30
40
50
60


E1M1 Episode Fitness Variance
Fitness
Generation
0
1000
2000
3000
4000
5000
0
10
20
30
40
50
60
Individual episodes
Elite average


E1M1 Parameter Evolution (normalized)
Value (0 = min, 1 = max of range)
0.8
0.6
0.4
0.2
0
10
20
30
40
50
60
Generation
loot_node_max_distance
stuck_recovery_ticks
combat_hold_ticks
health_threshold
armor_threshold
ammo_threshold
scan_interval


E1M1 Elite Fitness Stddev
Stddev (10-gen rolling)
Standard Deviation
Generation
0
200
400
600
800
1000
1200
1400
1600
10
20
30
40
50
60


E1M1 Challenger Win Rate
Win rate (10-gen rolling avg)
Challenger Win Rate
1.0
0.8
0.6
0.4
0.2
0.0
10
20
30
40
50
60
Generation


E1M1 Generation Timing
Seconds per gen
Mean: 79.0s
Seconds
Generation
200
175
150
125
100
75
50
25
0
10
20
30
40
50
60

Telemetry

Tier 1 Telemetry

Written at the end of an episode.

“ep_0001_summary.json”

Use for:

- GA fitness tracking
- Comparing genomes
- Post-run plots


```
"episode_id": 1,
  "level_id": "E1M1",
  "seed": 3297667121,
  "end_reason": "completion",
  "ticks": 3917,
  "health": 100.0,
  "armor": 5.0,
  "ammo": 40.0,
  "enemies_killed": 6,
  "waypoints_reached": 55,
  "stuck_events": 0,
  "damage_taken_total": 8,
  "ammo_used": 10,
  "fitness": 5258.69,
  "genome": {
    "loot_node_max_distance": 500,
    "stuck_recovery_ticks": 70,
    "combat_hold_ticks": 35,
    "health_threshold": 80,
    "armor_threshold": 5,
    "ammo_threshold": 25,
    "scan_interval": 140
```

Tier 0 Telemetry

Runs every tick of an episode.

“ep_0001_debug.jsonl”

Use for debugging specific runs

Currently only runs in ground mode.

Is having this and tier 2 necessary?

Both are per tick, but T0 has every detail.

T2 only has enough to recreate gameplay.


```
{"tick": 1,
"Unix_ms": 1774847917666,
"health": 100.0,
"armor": 0.0,
"ammo": 50.0,
"pos_x": 1056.0,
"pos_y": -3616.0,
"angle": 90.0,
"enemies_killed": 0,
"enemies_visible": 0,
"sm_state": "TRAVERSE",
"action": 4, "damage_taken":
0.0, "rss_mb": 38.4}
```

Tier 2 Telemetry

Runs every tick of an episode.

“ep_0001_actions.csv”

Action bitmask for recreating gameplay.

Positions + state for map recreation (next slide).

Need T1 and T2 to recreate the game currently. Is this correct?

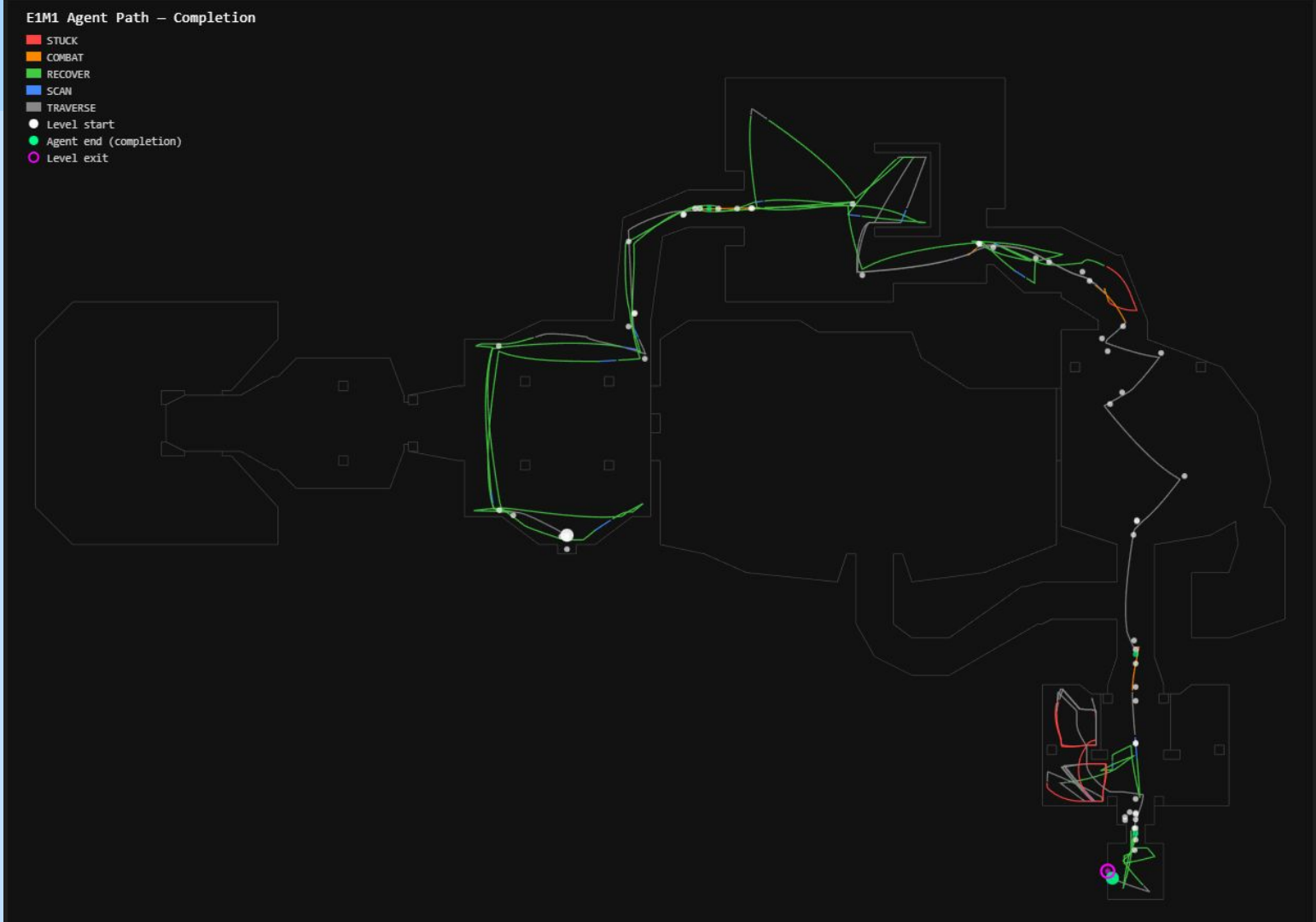
```csv
tick,action,sm_state,pos_x,pos_y
1,4,TRAVERSE,1056.0,-3616.0
2,4,TRAVERSE,1056.0,-3616.0
3,4,TRAVERSE,1056.0,-3616.0
4,4,TRAVERSE,1056.0,-3616.0
5,4,TRAVERSE,1056.0,-3616.0
6,4,TRAVERSE,1056.0,-3616.0
7,4,TRAVERSE,1056.0,-3616.0
8,4,TRAVERSE,1056.0,-3616.0
```

Tier 2 Telemetry

SVG is made from the other tier 2 csv

replay_map.py reads the CSV and draws the path.

Should T2 produce this automatically?



Codebase Improvements

Docker Support

- Assumes Ubuntu as Linux distro
- FSW can make the call on linux distro
- Only a few changes need to be made in Doom code

Files:

- .dockerignore
- .gitignore
- Dockerfile

Commands:

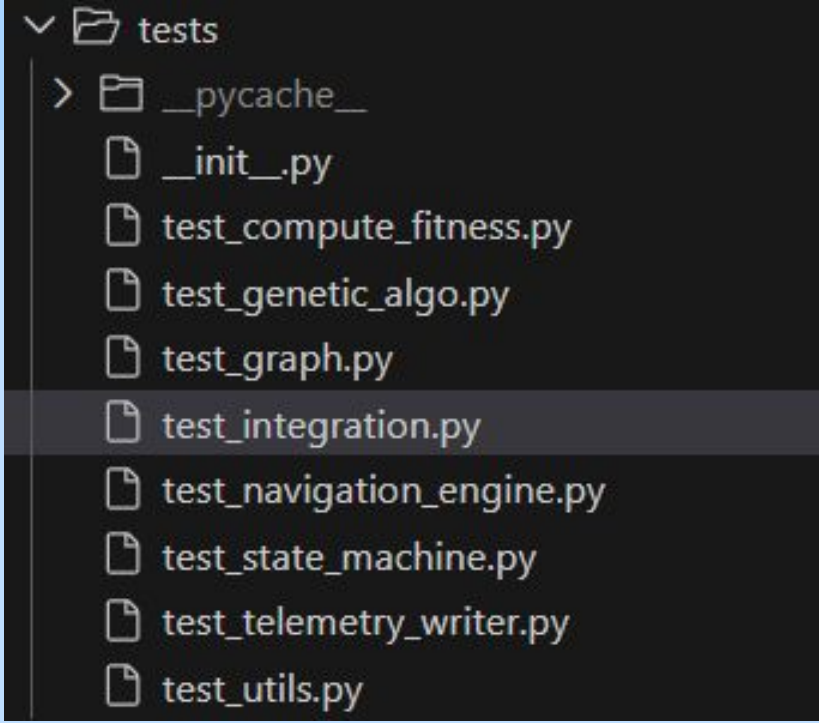
```
docker build -t doomsat .
docker run -v "$PWD/maps/wads/doom.wad:/app/maps/wads/doom.wad" doomsat
```

Dockerfile

```
#Ubuntu 22.04 assumed. distro not yet decided by flight software team.
#TODO: Update FROM line if the payload target changes.
FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive

#SDL_VIDEODRIVER=offscreen tells SDL2 to render without a display.
#This is cleaner than running Xvfb for pure headless operation.
#SDL_AUDIODRIVER=dummy suppresses audio init errors (Doom tries to open audio).
ENV SDL_VIDEODRIVER=offscreen
ENV SDL_AUDIODRIVER=dummy

#System dependencies required by VizDoom's pip wheel on Ubuntu 22.04.
#libsdl2-dev: SDL2 display/input backend
#libjpeg-dev, zlib1g-dev: image format support used by the engine
#libboost-all-dev, cmake: required if vizdoom builds from source as fallback
RUN apt-get update && apt-get install -y \
    python3 \
    python3-pip \
    libsdl2-dev \
    libjpeg-dev \
    zlib1g-dev \
    libboost-all-dev \
    cmake \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

#Install Python dependencies before copying source so this layer is cached
#as long as requirements.txt doesn't change.
COPY pyproject.toml .
RUN pip3 install --no-cache-dir .

COPY . .
```

Test Suite

- Unit tests and integration tests



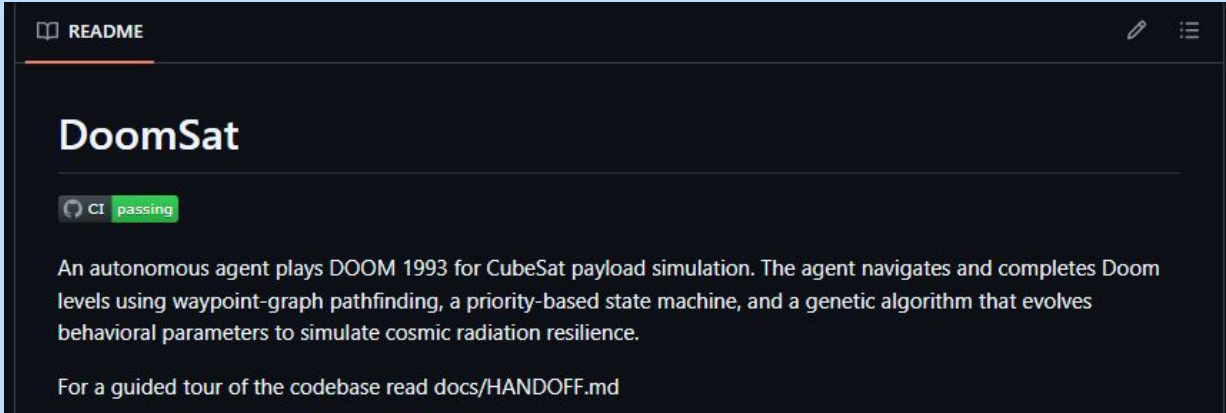

```
tests/test_utils.py::test_normalize_angle_large_negative PASSED                    [ 94%]
tests/test_utils.py::test_clear_line_no_segments PASSED                             [ 95%]
tests/test_utils.py::test_clear_line_no_obstruction PASSED                          [ 96%]
tests/test_utils.py::test_blocked_line PASSED                                       [ 97%]
tests/test_utils.py::test_clear_line_none_target PASSED                             [ 98%]
tests/test_utils.py::test_clear_line_collinear_touch PASSED                         [100%]

============================================================ 92 passed in 49.17s ============================================================
```

CI Pipeline

Only checks unit tests, not integration tests, for legal reasons.

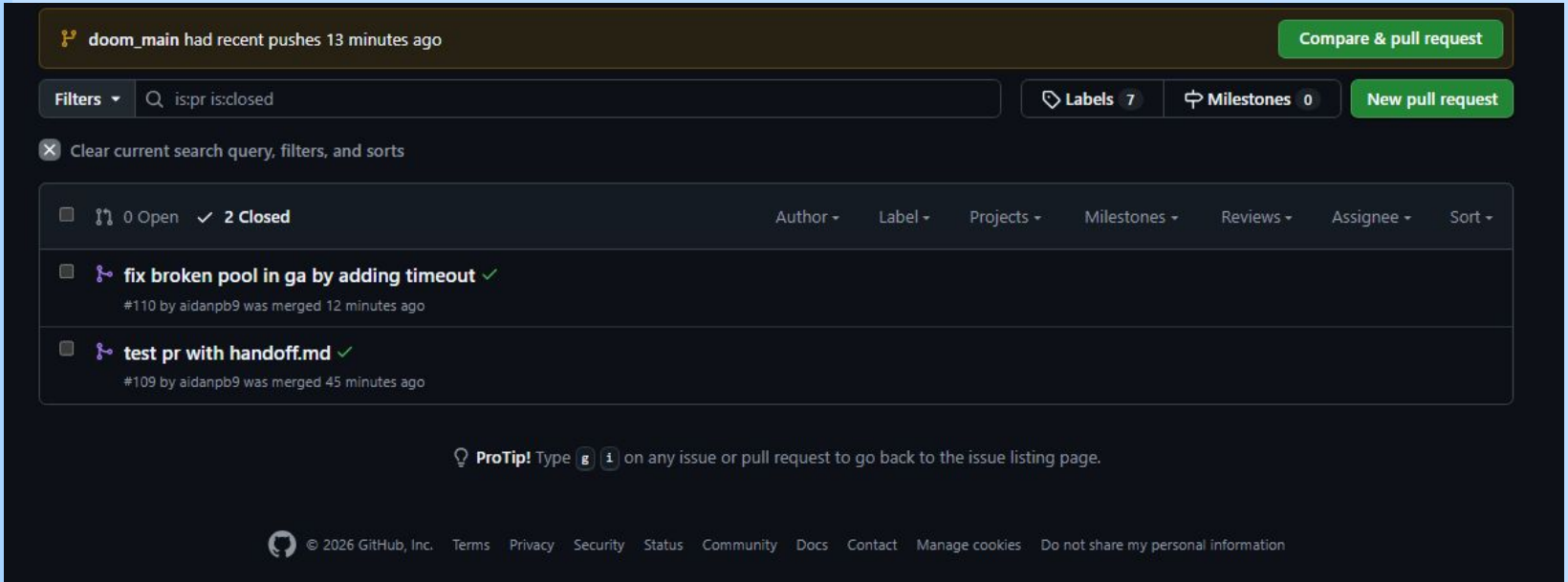
Runs on every push/merge to doom_main.

Contains a linter check too.



Merge Requests

- Keep the doom_main branch stable
- Owned by features, not users
- Don’t approve your own merge



The Readme

https://github.com/aidanpb9/DoomSat/tree/doom_main (link outdated after May 1, 2026)

Future Work

Execution Algo Bugs

Described in bugs/

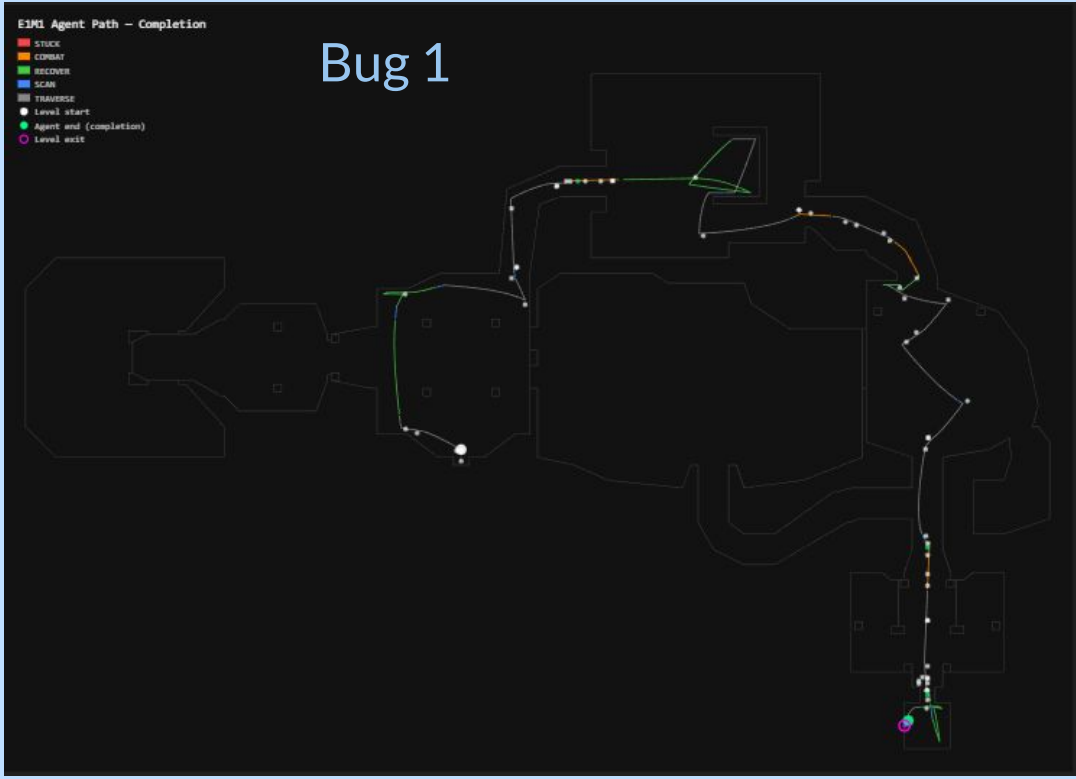
Bug1: weird behavior with state transitions and nodes.

Priority: medium

Bug2: agent can get caught in a loop re-targeting a node that it got stuck trying to visit

not observed yet, but possible

Priority: low



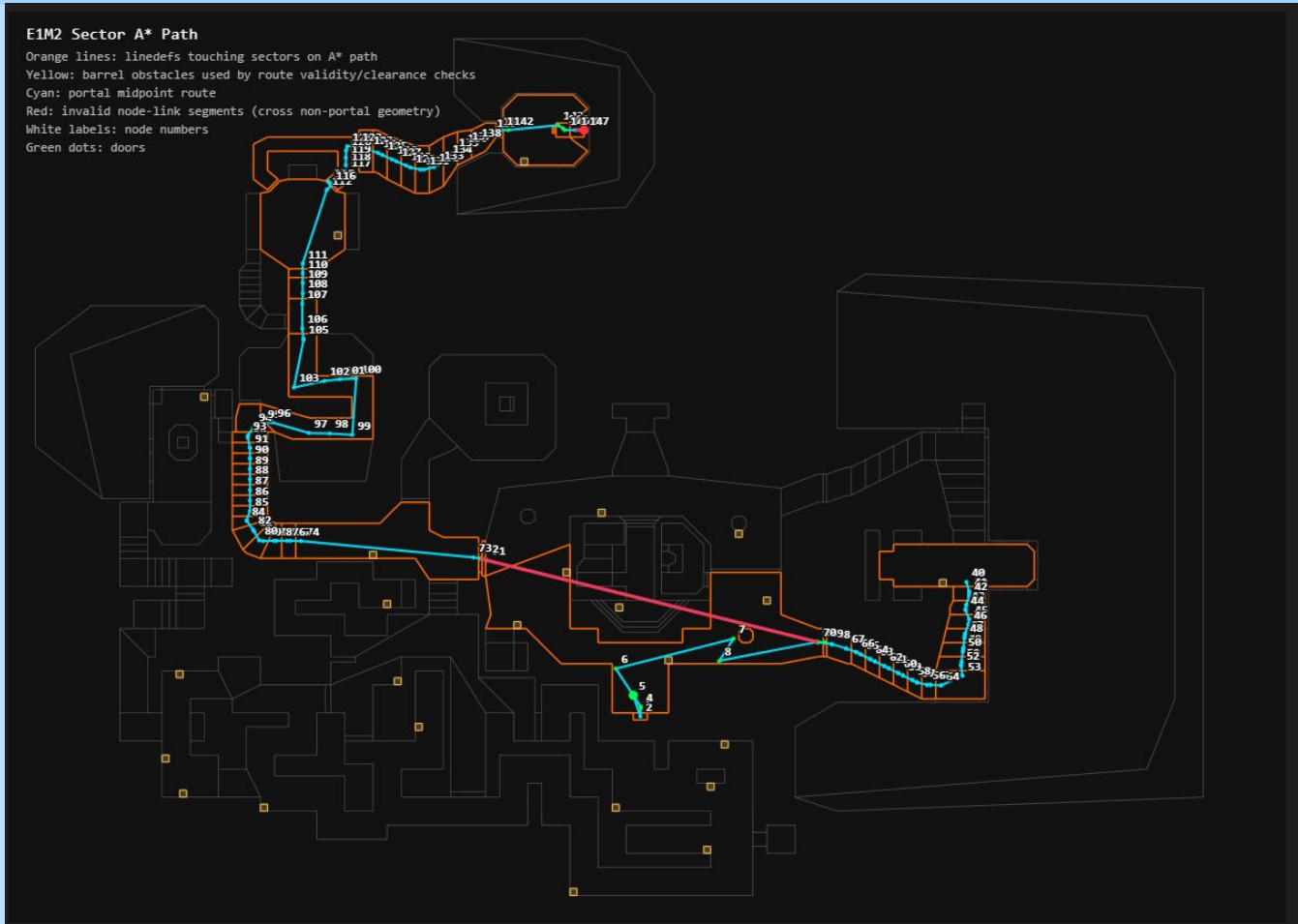
Improve navigation_planner.py

Produce valid paths for more levels.

Support E1M2 mechanics (keys, lifts, switches).

Needed for: execution algorithm

Priority: high



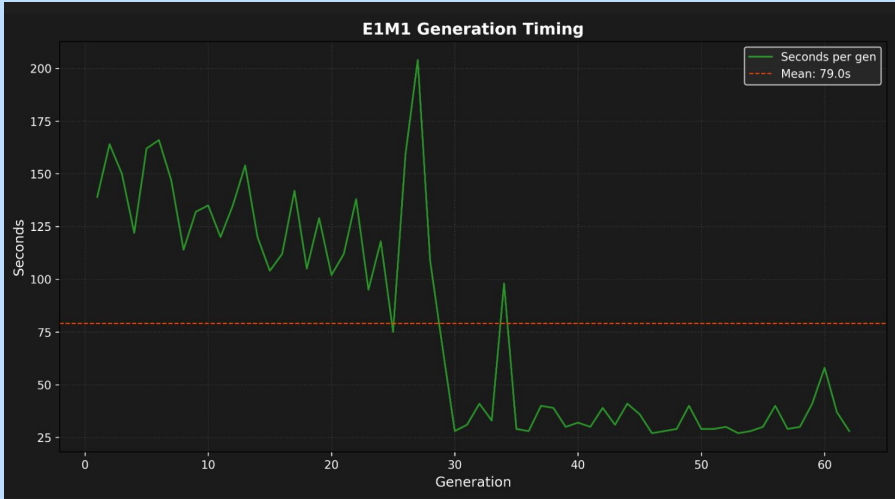
Improve GA runtime:

Currently we call init_game for every episode which adds overhead.

The reason it exists like this currently is because it's an easy way to reset the level.

Priority: Low

A better way to do it is have reset functions so we can avoid this overhead.
Could see a lot of improvement after this refactor.



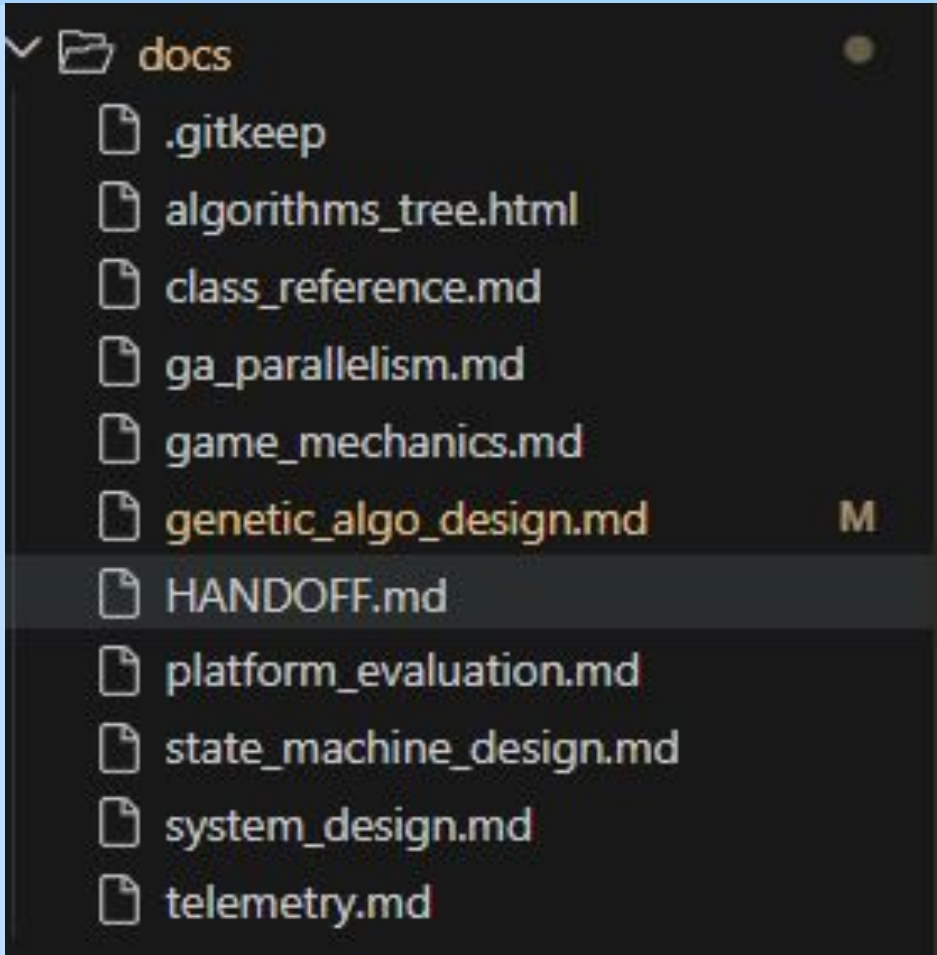
Handoff doc

Full documentation for existing code already.

Handoff is like a guide for the codebase

Need input from Aidan and Thomas for next group.

Priority: low (end of semester)



DOOM Guy Names

Rocky (inspired from Project Hail Mary)

Apollo

Halgorithm (Halgo for short)

Hal-9000

More Tasks

Showcase 4/2
Priority: MAX

Cleaning up experimental code and putting in a reference folder in main branch.
Priority: medium

Recreating game play from logs and testing determinism.
Priority: medium

GA plot to see how reward metrics are weighted
Need to normalize waypoints reached for the fitness function.
Priority: low

GA live visuals using TensorBoard.
Priority: low

FSW integration
Priority: MAX
Blocker: FSW

Should we make a research paper?